LongCat

Rep2Skill: Representation-Guided Skill Self-Evolution for LLM Agents

Kaixing Zhang[1], **Changming Li**[1,2], **Yingdong Shi**[1], **Zheng Zhang**[1,2]
Kaitao Song, **Wenjie Shi**[2], **Jingang Wang**[2], **Kan Ren**[1*]
[1]ShanghaiTech University [2]Meituan
[*]**Correspondence:** renkan@shanghaitech.edu.cn

arXiv:2609.39149v2 [cs.AI] 13 Oct 2026

Abstract

Textual skills enable large language model (LLM) based agents to accumulate reusable procedural knowledge without updating model parameters. Yet existing skill evolution remains largely confined to the text space: an optimizer must diagnose success and failure patterns, and revise skills solely from long execution trajectories and sparse task outcomes. This text-only paradigm leaves the agent's internal representations, which contain rich records of its evolving execution state, outside the skill optimization loop. *We ask whether an agent can improve its external textual skills by reflecting on its own internal representations.* We introduce Rep2Skill, a representation-guided framework for self-evolution on agent skills. Specifically, upon the collected agent rollouts, Rep2Skill models their internal model representation trajectories to localize turns that deviate from successful execution dynamics, and it further interprets these signals alongside the execution contexts as actionable textual feedback for targeted skill revision. Experiments on two widely-adopted agent environments with two open-source LLMs show that Rep2Skill consistently outperforms text-only approaches in the self-evolution setting, where the same LLM serves as both executor and optimizer without a stronger external model. This establishes a promising direction moving agent self-improvement beyond text-only reflection.

1 Introduction

Large Language Model (LLM) agents have increasingly demonstrated strong performance on multi-step, long-horizon tasks (Yao et al., 2023; Wang et al., 2023; Shridhar et al., 2021; Merrill et al., 2026). Skills, typically composed of reusable textual instructions, procedural rules, and task-solving strategies, provide agents with explicit guidance for accomplishing complex tasks (Anthropic, 2025; Zhou et al., 2026). Early agent skills were largely handcrafted by humans. To obtain higher quality skills at scale, recent work has explored skill evolution, where language model agents iteratively refine reusable textual skills from execution experience while keeping their model parameters fixed (Alzubi et al., 2026; Ni et al., 2026; Yang et al., 2026a; Tang et al., 2026).

Existing skill evolution methods predominantly rely on explicit textual artifacts, such as trajectory text (Ni et al., 2026; Yang et al., 2026a) and task-level outcomes (Alzubi et al., 2026). Despite their different optimization strategies, these approaches largely require the optimizer to infer from observable artifacts which parts of an execution contributed to failure, why they failed, and how the skill should be revised accordingly. This dependence places substantial burden on the LLM-based skill optimizer to attribute failures, diagnose their causes, and generate appropriate skill updates, especially when the optimizer model's capability is limited.

Meanwhile, an agent's observable trajectory only exposes its generated actions and environmental feedback, while leaving the *internal representation* underlying these decisions implicit. Recent mechanistic interpretability studies have increasingly shown that internal representations encode the model's behavioral information (Rimsky et al., 2024; Zou et al., 2023; Wang et al., 2025). Growing evidence suggests that an agent's internal representations contain information complementary to its observable execution trajectory (Yeh et al., 2026; Han et al., 2026). Yet, such representation signals remain largely outside the skill optimization loop. Therefore, we ask: *Can an agent improve its external textual skills by reflecting on its own internal representations?* We explore this by treating representation trajectories as an additional source of evidence for fine-grained credit assignment for attributing critical turns. In particular, by sampling multiple rollouts for the same task instance

under the same skill, we obtain a natural basis for comparison: differences in their representation dynamics can provide relative evidence for localizing failure-relevant turns and grounding more targeted skill revisions.

Building on these insights, we introduce Rep2Skill, a representation-guided framework that uses group-wise trajectory signals to provide fine-grained credit for textual skill evolution. It models internal representation dynamics and scores potentially failure-relevant turns within each rollout, then compares evidence across rollouts of the same task to guide skill revision. It then produces actionable textual feedback grounded in the corresponding trajectory context guided by these representation-space signals. Finally, the feedback is used to guide skill evolution in the text space, producing improved reusable skills for subsequent executions.

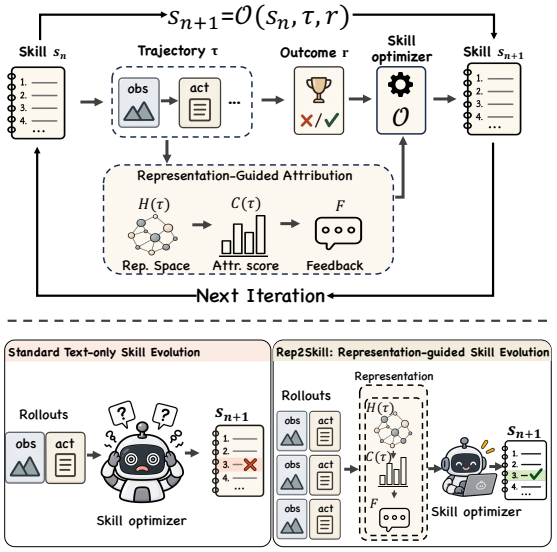


Figure 1 | Motivation of Rep2Skill. Top: Rep2Skill introduces internal representation signals into the iterative skill optimization loop to provide fine-grained guidance. Bottom: standard skill evolution relies on textual trajectories alone, while Rep2Skill leverages representation-guided evidence from grouped rollouts to support more targeted skill updates.

We evaluate Rep2Skill on two agentic benchmarks, ALFWorld (Shridhar et al., 2021) and WebShop (Yao et al., 2022), using two Qwen models (Yang et al., 2025; Qwen Team, 2026) to examine whether representation-guided credit assignment improves textual skill self-evolution and under what conditions such guidance is most beneficial. Across all settings, Rep2Skill consistently achieves the best downstream performance. Beyond end-task performance, we show that incorporating representation-space signals enables more fine-grained and accurate attribution of failure-relevant turns than text-only attribution. To better understand why group-wise representation-guided attribution benefits skill evolution, we analyze trajectory diversity within rollout groups from both the textual and representation perspectives, revealing substantial variation among executions sampled for the same task instance and skill. These findings support the need for group-wise sampling and relative trajectory comparison, and also demonstrate that representation space signals provide complementary evidence for guiding targeted skill refinement.

In summary, our contributions are threefold. **(i)** We introduce a novel perspective on textual skill evolution that leverages the model's internal states, showing that the representation space provides a complementary source for skill optimization beyond observable trajectory text. To the best of our knowledge, we are the first to leverage the inner representation of agent's trajectory for skill evolution. **(ii)** We propose Rep2Skill, a skill evolution framework that models representation trajectories and verbalizes representation space signals into useful textual guidance for skill improvement. **(iii)** We show that representation-guided feedback consistently improves skill evolution over text-only optimization on agentic benchmarks, with particularly large gains when the skill optimizer has limited capability.

2 Related Work

Experience-Driven Skill Evolution. Agent skills encode reusable procedural knowledge, enabling LLM agents to leverage prior experience in future tasks (Wang et al., 2023; Zhao et al., 2024; Liu et al., 2024; Wang et al., 2024; Fu et al., 2024). Recent frameworks allow agents to improve these skills by extracting and refining procedural knowledge from execution traces and outcome feedback (Xia et al., 2026; Mi et al., 2026; Yang et al., 2026b; Ding et al., 2026; Tang et al., 2026). These methods differ mainly in how experience is turned into skill edits: EvoSkill attributes failed rollouts to missing capabilities and revises modular skills accordingly (Alzubi et al., 2026). Trace2Skill aggregates trajectory-specific lessons across multiple executions into transferable skill documents (Ni et al., 2026), and SkillOpt treats skills as trainable textual states updated through bounded edits with validation-based selection (Yang et al., 2026a). Despite these differences, all of them derive their learning signal from externally observable artifacts such as trajectories, outcomes, and validation scores, which may result in sparse and delayed feedback and ambiguous failure attribution, while the model's internal representational state remains largely underexploited. Our Rep2Skill leverages internal representations to

provide fine-grained feedback for textual skill evolution, complementing the explicit execution artifacts used by prior approaches.

Representations of Large Language Models. Large language model's representation space encodes rich semantic and behavioral information, including truthfulness, factuality, reasoning-related properties (Tigges et al., 2023; Zou et al., 2023; Marks and Tegmark, 2023; Li et al., 2026). More recent studies further extend these observations to agentic settings, where internal representations have been shown to reflect tool-use decisions, execution failures, and affective or task-relevant states (Wu et al., 2026; Ruan et al., 2026; Yeh et al., 2026; Sofroniew et al., 2026). Such signals can often be detected from intermediate activations before they become explicit in the agent's observable trajectory, providing an additional source of evidence for understanding agent behavior (Ruan et al., 2026). Recent work has also leveraged model-internal emotion signals to guide skill selection in LLM agents (Lin et al., 2026). Beyond identifying information encoded in activation space, recent work has explored translating internal representations into natural language. Activation Oracles train language models to answer open-ended questions about activations, while Natural Language Autoencoders directly verbalize activation vectors through a natural-language bottleneck (Karvonen et al., 2026; Fraser-Taliente et al., 2026). Inspired by these latent-to-language approaches, we verbalize representation space signals into textual feedback, enabling model's internal information to directly guide textual skill evolution.

3 Method

We present REP2SKILL, a framework that leverages the agent's internal representations to guide textual skill evolution. As shown in Figure 2, given a task instance and the current skill, REP2SKILL performs *representation attribution* (Section 3.2) over sampled rollouts to identify informative turns and *verbalize their representation-space signals into textual feedback* (Section 3.3). The resulting feedback is then used to guide the skill optimizer toward targeted updates of the current skill (Section 3.4).

3.1 Preliminaries: Textual Skill Self-Evolution

Let π_θ denote a frozen LLM agent and s a textual skill that provides reusable procedural knowledge during execution. For a task instance x, executing the agent with skill s produces an interaction trajectory $\tau = (o_1, a_1, \ldots, o_T, a_T) \sim \pi_\theta(x; s)$, where o_t and a_t denote the observation and action at interaction turn t, respectively. The trajectory receives a task-level score $r(\tau) \in [0, 1]$. We define the expected performance of a skill on a dataset $\mathcal{D}$ as

$$J_{\mathcal{D}}(s) = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{\tau\sim\pi_\theta(x;s)}[r(\tau)]. \tag{1}$$

Standard skill evolution iteratively updates s to maximize $J_{\mathcal{D}}(s)$, with the optimization process primarily relying on observable textual trajectories and task-level outcomes.

$$s_{n+1} = \mathcal{O}\left(s_n, \tau\right), \tag{2}$$

where $\mathcal{O}$ denotes the textual skill optimizer. The optimization process therefore primarily relies on the observable interaction trajectory to identify useful experience and revise the skill toward improving $J_{\mathcal{D}}(s)$. Throughout this work, we consider a *self-evolution setting*, in which the same LLM serves as both the task executor π_θ and the textual skill optimizer $\mathcal{O}$, without relying on a stronger external model.

REP2SKILL augments this standard self-evolution process with the agent's internal representations produced during execution. At each interaction turn t, we extract a hidden state $\mathbf{h}_t \in \mathbb{R}^d$ from a fixed layer of the frozen agent, yielding the representation trajectory $\mathbf{H}(\tau) = (\mathbf{h}_1, \ldots, \mathbf{h}_T)$. The following sections describe how REP2SKILL obtains representation-space attribution, verbalizes it into grounded feedback, and incorporates the resulting evidence into skill evolution.

3.2 Attribution in Representation Space

Textual skill evolution faces two fundamental challenges when learning from agent experience: *coarse task-level outcomes* and *diverse agentic rollouts*. Final success or failure provides little guidance about which interaction turns should receive credit for the observed result. Even under the same task instance and textual skill, sampled trajectories can differ substantially, making skill revisions based on a single rollout sensitive to incidental behaviors. Recent studies suggest that language models' internal representations encode fine-grained signals about agent progress and failure-relevant states, providing a promising source of evidence for localizing

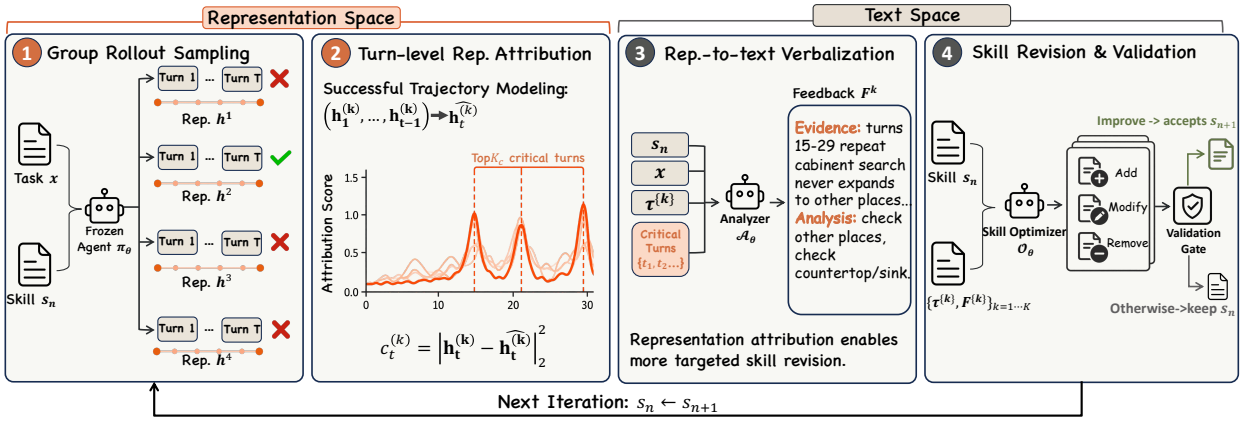


Figure 2 | Overview of REP2SKILL. Given grouped rollouts under the current skill s_n, a trajectory model trained on successful executions scores each turn by its representation prediction error, and the top-K_c critical turns are verbalized into textual feedback $F^{(k)}$. The optimizer then revises s_n with this feedback through a validation-gated update. The same frozen LLM π_θ serves as executor, analyzer and optimizer.

credit-critical interaction turns (Yeh et al., 2026; Ruan et al., 2026). Together, these challenges motivate our group-wise representation attribution, which derives fine-grained turn-level credit from internal representations while leveraging multiple rollouts sampled from one query for more robust diagnostic evidence.

Grouped rollout sampling. To capture the behavioral variation of LLM agent's trajectory, we organize experience into groups of rollouts generated for the same task instance under the same textual skill. Inspired by group-relative optimization (Shao et al., 2024), this design uses within-group variation as structured evidence for skill diagnosis: the group exposes diverse execution behaviors while providing a controlled reference under a shared task and skill context. Formally, for each task instance x and current textual skill s_n, we sample a group of K rollouts from the same agent:

$$\mathcal{G}(x, s_n) = \left\{\tau^{(k)} \sim \pi_\theta(\cdot \mid x, s_n)\right\}_{k=1}^{K}, \tag{3}$$

where $\tau^{(k)} = \{(o_t^{(k)}, a_t^{(k)})\}_{t=1}^{T_k}$ denotes the k-th interaction trajectory. The resulting group broadens the behavioral evidence available for skill revision and further provides a natural reference for the representation-space diagnosis introduced next.

Turn attribution in representation space. For each rollout $\tau^{(k)}$, we extract the hidden state $\mathbf{h}_t^{(k)} \in \mathbb{R}^d$ at every interaction turn from a fixed layer of the frozen agent, yielding its representation trajectory $\mathbf{H}(\tau^{(k)}) = (\mathbf{h}_1^{(k)}, \ldots, \mathbf{h}_{T_k}^{(k)})$. To improve generalizability and reduce computational cost, we project each $\mathbf{h}_t^{(k)}$ to a low-dimensional space with PCA, and with a slight abuse of notation we still denote the projected representation by $\mathbf{h}_t^{(k)}$. We adopt an autoregressive sequence model (Yeh et al., 2026) to capture the temporal evolution of these states. At each turn t, the model uses the preceding representation history $(\mathbf{h}_1^{(k)}, \ldots, \mathbf{h}_{t-1}^{(k)})$ to predict $\widehat{\mathbf{h}}_t^{(k)}$, the representation expected at the next turn under the dynamics learned from successful executions.

Specifically, we utilize a Neural Controlled Differential Equation (Neural CDE) (Kidger et al., 2020) model which has been trained on successful agent rollouts, to depict the latent trajectory with the LLM. At inference time, we use the prediction error $c_t^{(k)} = \|\mathbf{h}_t^{(k)} - \widehat{\mathbf{h}}_t^{(k)}\|_2^2$ as the turn-level representation attribution score, where a larger value indicates a stronger deviation from successful-execution dynamics. We select the top-K_c turns with the highest attribution scores as critical turns and pass them to the skill optimizer as localized credit-assignment signals for targeted skill revision.

3.3 Representation-to-Text Verbalization

Critical turn selection. The representation attribution scores localize informative turns, but do not directly specify how the textual skill should be revised. We use the same frozen LLM as an analyzer, denoted by $\mathcal{A}_\theta$, to verbalize representation-space attribution into textual feedback. For the k-th rollout $\tau^{(k)}$ in a group, let

$\mathbf{c}^{(k)} = (c_1^{(k)}, \ldots, c_{T_k}^{(k)})$ denote its turn-level attribution scores. We select the top-K_c turns with the highest scores as the critical turn set $\mathcal{C}^{(k)} = \mathrm{TopK}(\mathbf{c}^{(k)}, K_c)$, and provide them to the analyzer $\mathcal{A}_\theta$ together with the current skill s_n, the task instance x, and the complete trajectory $\tau^{(k)}$. The selected turns provide localized representation evidence, while the full trajectory and current skill provide the context needed to interpret their implications for skill revision.

Representation-guided feedback generation. Since a high attribution score only indicates deviation from successful execution dynamics, it does not necessarily imply an erroneous skill rule. We therefore require the analyzer $\mathcal{A}_\theta$ to ground each attribution in explicit evidence from the trajectory and current skill, producing the textual feedback

$$F^{(k)} = \mathcal{A}_\theta \left(s_n, x, \tau^{(k)}, \left\{ (t, c_t^{(k)}) \mid t \in \mathcal{C}^{(k)} \right\} \right), \tag{4}$$

where $F^{(k)}$ denotes the feedback derived from rollout $\tau^{(k)}$. Each feedback item identifies the relevant execution context, its relation to the existing skill, and a potential revision when supported by sufficient evidence. The analyzer further concludes whether the behavior is already covered by the current skill or whether the evidence is insufficient for revision.

As shown in Section 4.3, incorporating representation-based attribution enables more accurate fine-grained credit assignment than relying on trajectory text alone, supporting its use for generating targeted skill feedback. The validated feedback $\{F^{(k)}\}_{k=1}^{K}$ is then provided to the textual skill optimizer $\mathcal{O}$ for updating s_n.

3.4 Skill Evolution in Text Space

Representation-Guided Skill Revision. Given the textual feedback produced by the analyzer, the skill optimizer revises the current skill s_n by consolidating the identified issues and corresponding revision suggestions into a set of candidate skills (Figure 2, step 4). Specifically, given the group of agent rollouts and their representation-guided feedback, the candidate skills are generated as

$$s_{n+1} = \mathcal{O}_\theta \left(s_n, \{\tau^{(k)}, F^{(k)}\}_{k=1}^{K} \right). \tag{5}$$

Compared with standard textual skill evolution in Equation (2), the optimizer additionally consolidates representation-guided feedback from the grouped rollouts to guide the revision.

Following Yang et al. (2026a), we adopt a consolidation and validation-gated update procedure to ensure reliable revisions. Implementation details are provided in Section B.2. Each candidate is evaluated on a small validation set, and the best-performing candidate is accepted only if it improves over the current skill. Otherwise, s_n is retained for the next iteration.

4 Experiment

Our experiments investigate three primary research questions (**RQs**): **RQ1:** Does representation-guided skill evolution improve agent performance over existing textual skill evolution methods? **RQ2:** How does representation contribute to effective skill evolution? **RQ3:** When and why is Rep2Skill effective?

4.1 Experimental Setup

Evaluation Benchmarks. We evaluate skill-evolution methods on two representative agentic benchmarks: ALFWorld (Shridhar et al., 2021) and WebShop (Yao et al., 2022), covering embodied household tasks and interactive web-based shopping tasks, respectively.

Baselines. We include No Skill as a baseline and further compare Rep2Skill with several representative skill-evolution methods, including Trace2Skill (Ni et al., 2026), EvoSkill (Alzubi et al., 2026), and SkillOpt (Yang et al., 2026a).

Implementation Details. Following (Tang et al., 2026; Ni et al., 2026), all methods are evaluated under a self-evolution setting, where skill evolution relies solely on the target model itself, without feedback, or supervision from a larger external model. To account for stochasticity in agent trajectories, we report results averaged over three independent runs with different random seeds. For ALFWorld, we evaluate success rate on

Table 1 | Self-evolving skill performance on ALFWorld and WebShop. All results are mean ± standard deviation over 3 independent evolution runs.

Model	Method	ALFWorld	WebShop	
		Succ (%) ↑	Score ↑	Succ (%) ↑
Qwen3-4B	No Skill	22.64±1.76	41.86±1.65	13.67±1.53
	Trace2Skill (Ni et al., 2026)	47.26±5.18	45.68±3.40	13.00±1.41
	EvoSkill (Alzubi et al., 2026)	43.03±3.52	49.48±2.02	13.33±0.94
	SkillOpt (Yang et al., 2026a)	46.51±3.12	39.04±2.49	11.67±3.40
	Rep2Skill (Ours)	**58.21**±2.66	**49.79**±5.95	**16.33**±3.51
Qwen3.5-9B	No Skill	28.61±1.55	50.07±2.66	18.33±0.94
	Trace2Skill (Ni et al., 2026)	52.74±6.25	24.77±3.91	12.00±1.63
	EvoSkill (Alzubi et al., 2026)	59.45±3.13	33.10±8.06	15.67±3.77
	SkillOpt (Yang et al., 2026a)	62.19±2.28	52.12±2.22	18.67±1.25
	Rep2Skill (Ours)	**69.90**±2.88	**53.56**±2.52	**23.33**±3.30

the unseen evaluation split. For WebShop, we report the average task score and success rate. For each query, we sample 4 rollouts as a group for optimization. We conduct experiments with two open-source models from the Qwen family: Qwen3-4B (Yang et al., 2025) and Qwen3.5-9B (Qwen Team, 2026). This selection covers different model scales, architectures (full attention v.s. hybrid attention) highlighting the generalizability of our method.

4.2 Representation-Guided Skill Evolution (RQ1)

We first evaluate whether representation-guided credit assignment leads to more effective skill evolution across downstream agent tasks.

Rep2Skill consistently improves skill evolution. As shown in Table 1, Rep2Skill achieves the highest mean performance on both benchmarks with both models. On ALFWorld, it reaches success rates of 58.21% with Qwen3-4B and 69.90% with Qwen3.5-9B, exceeding the strongest textual skill-evolution baselines by 10.95 and 7.71 percentage points, respectively. On WebShop, Rep2Skill achieves success rates of 16.33% and 23.33%, alongside task scores of 49.79 and 53.56, with Qwen3-4B and Qwen3.5-9B, respectively. The gains are especially clear with Qwen3.5-9B, where Rep2Skill exceeds the strongest baseline by 4.66 points in success rate and 1.44 points in task score. These consistent gains indicate that the use of representation-guided feedback provides more informative guidance for subsequent textual skill revision, leading to more effective self-evolution across tasks and model scales.

Representation guidance makes skill evolution more reliable. Under the self-evolution setting, the optimizer model may not always be capable of extracting useful and generalizable experience from sampled rollouts, especially when the underlying model is relatively limited in capability. As a result, textual skill evolution does not always improve over the original agent. On WebShop with Qwen3-4B, SkillOpt yields a lower task score than No Skill (39.04 versus 41.86), while Rep2Skill improves both success rate and task score relative to No Skill. With Qwen3.5-9B, Trace2Skill and EvoSkill also underperform No Skill on both WebShop metrics. This pattern suggests that feedback derived from trajectories alone can sometimes lead to ineffective skill revisions, while representation-guided attribution provides additional evidence for selecting execution steps that inform the update.

4.3 Why Does Representation Help Skill Evolution? (RQ2)

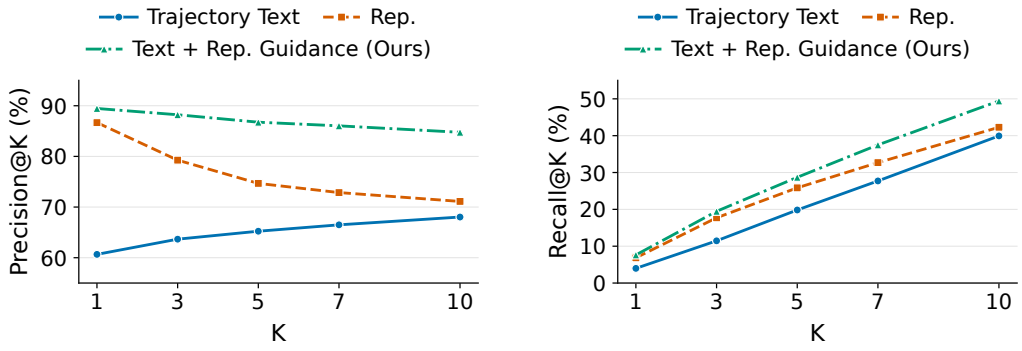
To understand why representation guidance improves downstream skill evolution, we examine whether internal representation trajectories provide reliable signals for identifying critical turns.

Evaluation Setup. We formulate turn-level attribution as a non-progress turn prediction problem. Each interaction turn is labeled by GPT6-Astra (OpenAI, 2026) as exhibiting positive, neutral, or negative progress toward task completion, and we treat neutral and negative turns as non-progress turns. We evaluate on 100 ALFWorld trajectories. For trajectory-level aggregation, we compute each metric separately within each

trajectory and then report the macro-average across trajectories. Given a rollout, the model ranks individual interaction turns according to their likelihood of being non-progress turns. We compare three sources of attribution evidence: (1) *Trajectory Text*, where the LLM directly identifies non-progress turns from the whole rollout, (2) *Rep. Score*, where turn-level scores are obtained solely from representation trajectories, and (3) *Text + Rep. Guidance*, our setting, where a representation analyzer interprets the trajectory modeling signals into textual patterns, which are then provided alongside the rollout to the same attribution LLM. Both the execution model and analyzer model are Qwen3-4B (Yang et al., 2025). We choose $K_c = 3$ in Table 2.

Table 2 | Critical-turn detection under different attribution signals. Representation-space signals provide complementary information to trajectory text. Both the execution model and analyzer model are Qwen3-4B (Yang et al., 2025).

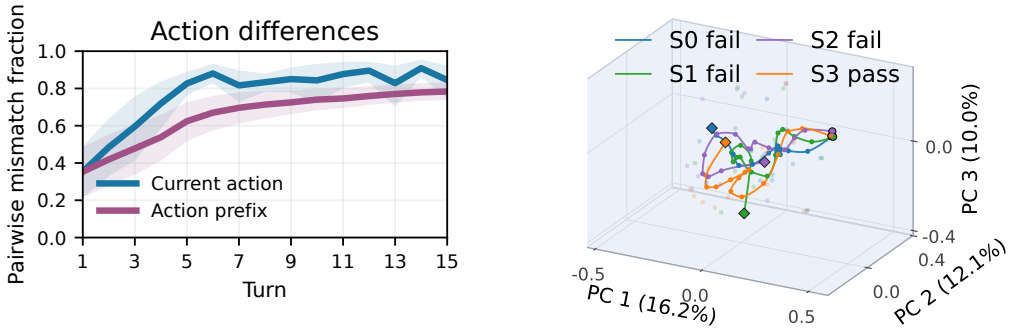
Input	AUROC ↑	AUPRC ↑	P@Top-3 ↑
Trajectory Text	0.494	0.736	63.66%
Rep.	0.592	0.789	79.26%
Text + Rep. (**Ours**)	**0.838**	**0.876**	**88.22%**



(a) Precision@K. (b) Recall@K.

Figure 3 | Turn-level non-progress detection with different attribution inputs. Precision@K and Recall@K are macro-averaged over trajectories for $K \in \{1, 3, 5, 7, 10\}$. Text + Rep. Guidance achieves the highest precision and recall.

Representation signals enable reliable turn-level attribution. As shown in Table 2, trajectory text alone provides limited discrimination of non-progress turns. Representation scores derived solely from internal representation trajectories exhibit stronger turn-level signals, improving AUROC to 0.592. More importantly, augmenting the trajectory with representation-derived guidance substantially improves the same analyzer LLM, achieving 0.838 AUROC, 0.876 AUPRC, and 88.22% Precision@Top-3. This large improvement indicates that representation can provide useful guidance for interpreting which parts of a trajectory fail to make positive progress.



(a) Text-space diversity. (b) Representation-space diversity.

Figure 4 | Diversity of trajectories in text and representation space. Within-query action diversity and representation trajectories for four rollouts of one ALFWorld task.

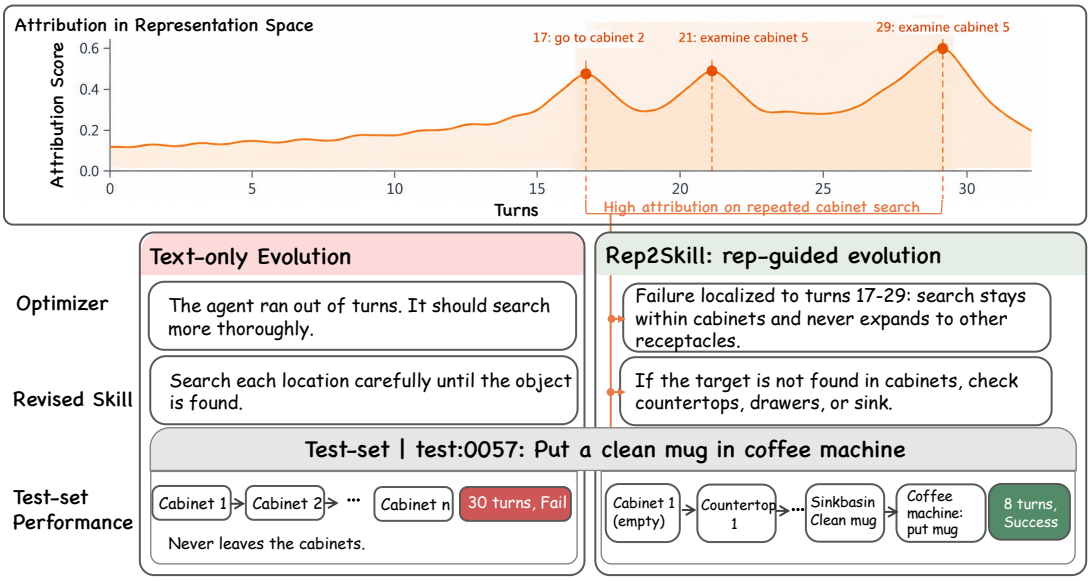


Figure 5 | Case Study of Rep2Skill. Attribution localizes the failure to repeated cabinet search, turning a generic reflection into a concrete, reusable skill rule.

4.4 Further Analyses of Rep2Skill (RQ3)

Having established that representation signals provide more reliable fine-grained credit for skill evolution, we next examine the key design choices and underlying properties of Rep2Skill through further analyses and discussion.

Both text-space and representation-space's trajectory show diversity within same-query rollout groups. We analyze 84 groups of Qwen3-4B rollouts on 12 ALFWorld tasks under an empty skill. Figure 4a shows pairwise action disagreement at each turn and averaged over the prefix through that turn. Current-turn disagreement reaches 82.7% at turn 5, indicating that rollouts within a group differ in their action choices despite sharing the task and skill. Figure 4b illustrates four rollouts of one task with mixed final outcomes. Their first 15 turns follow distinct paths in a shared PCA space, highlighting the complementary evidence available within a rollout group.

Qualitative Analysis: Representation attribution enables more targeted skill revision. Figure 5 illustrates how representation-guided analysis can lead to a more precise skill update from the same failed trajectory. The text-only optimizer mainly observes the terminal symptom that the agent exhausts its interaction budget, and consequently proposes a generic revision encouraging more thorough search. This feedback fails to localize the critical turns and identify the underlying search pattern, causing the revised skill to reproduce essentially the same behavior and fail again after 30 steps. In contrast, representation attribution assigns consistently high scores to the later segment of the trajectory, with prominent peaks around repeated cabinet-search actions. The representation analyzer uses these localized signals together with the trajectory context to identify a more specific failure mode, which supports a targeted skill revision that explicitly instructs the agent to expand its search to countertops, drawers, or the sink when cabinets are exhausted. With this revised skill, the agent discovers the clean mug in the sink basin and completes the task in 8 steps. The example shows that representation signals can expose localized execution patterns that are difficult to infer from task failure alone.

Table 3 | Cumulative ablation on ALFWorld. Components are removed progressively from top to bottom, and ↓ denotes the drop relative to the previous row. We report the success rate (SR), averaged over three independent runs.

Variant	ALFWorld SR (%)
Rep2Skill (full)	**58.21**
– rep. verbalization	56.47 (↓1.74)
– rep. attribution	50.75 (↓5.72)
– group-wise rollout evidence	46.51 (↓4.24)
No Skill	22.64

Ablation. Complementary to the above analysis, we further examine the contribution of individual components in Rep2Skill on ALFWorld while keeping the rollout and optimizer budgets fixed. As shown in Table 3, the full Rep2Skill achieves the best performance among all variants. Removing representation verbalization while retaining representation attribution reduces the success rate, suggesting that localized representation signals become more useful when they are interpreted into trajectory-grounded textual feedback before skill revision. Removing representation attribution entirely causes a further degradation: under this variant, the optimizer still observes the same grouped rollouts but receives no representation-derived evidence. This result suggests that the observed improvement cannot be explained solely by grouped trajectory comparison and supports the complementary value of internal representation signals for skill evolution. Finally, removing group-wise rollout evidence also reduces performance, indicating that relative comparison across executions provides a useful basis for identifying informative representation deviations. Overall, these ablations show that the performance gains of Rep2Skill rely on three complementary ingredients: group-wise comparison, representation-space attribution, and semantic interpretation of the attributed evidence.

5 Conclusion

In this work, we investigated whether LLM agents can improve their external textual skills by reflecting on their *own* internal representations. We introduced Rep2Skill, which brings representation trajectories within the model into the skill-evolution loop, by grounding inner-model representation signals to the deviation from successful execution dynamics for downstreaming skill revision. Experiments on two widely adopted benchmarks ALFWorld and WebShop with two open-source LLMs show that representation guidance leads to more effective skill evolution than text-only feedback. Turn-level analysis and component ablations further support the respective roles of representation guidance in skill self-evolution. More broadly, Rep2Skill moves agent's improvement beyond text-only reflection toward more comprehensive self-evolution that couples representation-space analysis with text-space evolution.

Limitations

Our method requires access to the language model's internal representations, which may limit its direct applicability to white-box models. For black-box models, using an accessible proxy model to provide representation signals could be a feasible alternative. Moreover, our experiments are primarily conducted on models from the Qwen family. Extending the evaluation to a broader range of model architectures would help establish the broader generality of the method. In addition, representation trajectory modeling introduces modest extra overhead, including approximately 50 additional rollouts and a lightweight training stage.

References

Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Christian Alexander Calvo, Weiyuan Chen, and Tu Vu. 2026. Evoskill: Automated skill discovery for multi-agent systems. In *Third Conference on Language Modeling*.

Anthropic. 2025. Equipping agents for the real world with agent skills. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills. Anthropic Engineering Blog.

Ruomeng Ding, Wei Cheng, Minglai Shao, and Chen Zhao. 2026. Skillgen: Learning domain skills for in-context sequential decision making. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 30512–30520.

Kit Fraser-Taliente, Subhash Kantamneni, Euan Ong, Dan Mossing, Christina Lu, Paul C Bogdan, Emmanuel Ameisen, James Chen, Dzmitry Kishylau, Adam Pearce, et al. 2026. Natural language autoencoders produce unsupervised explanations of llm activations. *Transformer Circuits Thread*.

Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. Autoguide: Automated generation and selection of context-aware guidelines for large language model agents. *Advances in Neural Information Processing Systems*, 37:119919–119948.

Jiahui Han, Qinuo Li, Ziheng Peng, Haotian Wu, Haoze Liu, Danfeng Shan, Guanchu Wang, Huiqi Deng, and Ninghao Liu. 2026. Skilleval: Decomposing agent skill quality into interpretable signals. *arXiv preprint arXiv:2608.06891*.

Adam Karvonen, James Chua, Clément Dumas, Kit Fraser-Taliente, Subhash Kantamneni, Julian Minder, Euan Ong, Arnab Sen Sharma, Daniel Wen, Owain Evans, and Samuel Marks. 2026. Activation oracles: Training and evaluating LLMs as general-purpose activation explainers. In *Forty-third International Conference on Machine Learning*.

Patrick Kidger, James Morrill, James Foster, and Terry Lyons. 2020. Neural controlled differential equations for irregular time series. In *Advances in Neural Information Processing Systems*, volume 33, pages 6696–6707. Curran Associates, Inc.

Changming Li, Kaixing Zhang, Haoyun Xu, Yingdong Shi, Zheng Zhang, Kaitao Song, and Kan Ren. 2026. Interpreting and controlling llm reasoning through integrated policy gradient. *arXiv preprint arXiv:2602.02313*.

Bohan Lin, Hejia Geng, Xinyi Xie, Heng Zhou, Qinghua Xing, Bo Liu, Chen Zhang, and Yudong Zhang. 2026. Emotion2skill: Model-internal emotion signals for adaptive skill selection and evolution. *Preprint*, arXiv:2608.09248.

Anthony Zhe Liu, Jongwook Choi, Sungryull Sohn, Yao Fu, Jaekyeom Kim, Dong-Ki Kim, Xinhe Wang, Jaewon Yoo, and Honglak Lee. 2024. Skillact: Using skill abstractions improves llm agents. In *ICML 2024 Workshop on LLMs and Cognition*.

Samuel Marks and Max Tegmark. 2023. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. *arXiv preprint arXiv:2310.06824*.

Mike A Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, and 65 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*.

Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. 2026. Skillpro: Learning reusable skills from experience via non-parametric ppo for llm agents. *arXiv preprint arXiv:2602.01869*.

Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. 2026. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*.

OpenAI. 2026. GPT-6 Astra: A New Generation of Intelligence. https://openai.com/index/gpt-6-astra/. Accessed: 2026-09-18.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents.

Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Turner. 2024. Steering llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15504–15522, Bangkok, Thailand. Association for Computational Linguistics.

Kai Ruan, Zihe Huang, Ziqi Zhou, Qianshan Wei, Jinghao Lin, Xuan Wang, and Hao Sun. 2026. Doomed from the start: Early abort of llm agent episodes via a recall-controlled probe cascade. *arXiv preprint arXiv:2607.06503*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. {ALFW}orld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.

Nicholas Sofroniew, Isaac Kauvar, William Saunders, Runjin Chen, Tom Henighan, Sasha Hydrie, Craig Citro, Adam Pearce, Julius Tarng, Wes Gurnee, et al. 2026. Emotion concepts and their function in a large language model. *arXiv preprint arXiv:2604.07729*.

Liyan Tang, Cyrus Rashtchian, Chun-Sung Ferng, Andrew Tomkins, Da-Cheng Juan, and Tu Vu. 2026. Wikiskill: Compiling agent experience into persistent knowledge for skill evolution. *arXiv preprint arXiv:2608.27454*.

Curt Tigges, Oskar John Hollinsworth, Atticus Geiger, and Neel Nanda. 2023. Linear representations of sentiment in large language models. *arXiv preprint arXiv:2310.15154*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. In *Intrinsically-Motivated and Open-Ended Learning Workshop @NeurIPS2023*.

Keyu Wang, Jin Li, Shu Yang, Zhuoran Zhang, and Di Wang. 2025. When truth is overridden: Uncovering the internal origins of sycophancy in large language models. *Preprint*, arXiv:2508.02087.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent workflow memory. *arXiv preprint arXiv:2409.07429*.

Zekun Wu, Ze Wang, Seonglae Cho, Yufei Yang, Adriano Koshiyama, Sahan Bulathwela, and Maria Perez-Ortiz. 2026. Tool calling is linearly readable and steerable in language models. In *Workshop on Failure Modes of Agentic AI at ICML 2026*.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. 2026. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, et al. 2026a. Skillopt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*.

Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. 2026b. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Samuel Yeh, Yiwen Zhu, Shaleen Deep, and Sharon Li. 2026. Tracing agentic failure from the flow of success. *arXiv preprint arXiv:2607.12747*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.

Yingli Zhou, Wang Shu, Yaodong Su, Wenchuan Du, Yixiang Fang, and Xuemin Lin. 2026. A comprehensive survey on agent skills: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2605.07358*.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. 2023. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*.

A Algorithms of REP2SKILL

In this section, we provide the main algorithms of our method in order to clearly illustrate the key steps of REP2SKILL, summarized in Algorithm 1.

Algorithm 1 REP2SKILL: Representation-Guided Skill Self-Evolution

Require: Frozen LLM agent π_θ (also used as analyzer $\mathcal{A}_\theta$ and optimizer $\mathcal{O}_\theta$); initial skill s_0; training set $\mathcal{D}_{\text{train}}$; validation set $\mathcal{D}_{\text{val}}$; number of iterations N; group size K; number of critical turns K_c
Ensure: Evolved skill s_N

Train the representation trajectory model (Neural CDE) on PCA-projected representation trajectories $\mathbf{H}(\tau)$ of successful rollouts, with PCA fitted on the same rollouts
for $n = 0, 1, \ldots, N-1$ **do**
  Sample a batch of task instances $\mathcal{B} \subset \mathcal{D}_{\text{train}}$; initialize $\mathcal{E} \leftarrow \emptyset$
  for each task instance $x \in \mathcal{B}$ **do**
    // Group-wise representation attribution (Section 3.2)
    Sample a group of rollouts $\mathcal{G}(x, s_n) = \{\tau^{(k)} \sim \pi_\theta(\cdot \mid x, s_n)\}_{k=1}^{K}$
    for $k = 1, \ldots, K$ **do**
      Extract hidden states $\mathbf{H}(\tau^{(k)}) = (\mathbf{h}_1^{(k)}, \ldots, \mathbf{h}_{T_k}^{(k)})$ from a fixed layer and project them with PCA
      for $t = 2, \ldots, T_k$ **do**
        Predict $\widehat{\mathbf{h}}_t^{(k)}$ from the history $(\mathbf{h}_1^{(k)}, \ldots, \mathbf{h}_{t-1}^{(k)})$
        Compute attribution score $c_t^{(k)} \leftarrow \|\mathbf{h}_t^{(k)} - \widehat{\mathbf{h}}_t^{(k)}\|_2^2$
      // Representation-to-text verbalization (Section 3.3)
      Select critical turns $\mathcal{C}^{(k)} \leftarrow \mathrm{TopK}(\mathbf{c}^{(k)}, K_c)$
      Generate feedback $F^{(k)} \leftarrow \mathcal{A}_\theta\big(s_n, x, \tau^{(k)}, \{(t, c_t^{(k)}) \mid t \in \mathcal{C}^{(k)}\}\big)$
    $\mathcal{E} \leftarrow \mathcal{E} \cup \{(\tau^{(k)}, F^{(k)})\}_{k=1}^{K}$
  // Skill evolution in text space (Section 3.4)
  Generate candidate skills $\{\tilde{s}_j\}_{j=1}^{M} \leftarrow \mathcal{O}_\theta(s_n, \mathcal{E})$ via consolidation
  $s^\star \leftarrow \arg\max_j J_{\mathcal{D}_{\text{val}}}(\tilde{s}_j)$
  if $J_{\mathcal{D}_{\text{val}}}(s^\star) > J_{\mathcal{D}_{\text{val}}}(s_n)$ **then**
    $s_{n+1} \leftarrow s^\star$ ▷ Accept the improved skill
  else
    $s_{n+1} \leftarrow s_n$ ▷ Retain the current skill
return s_N

B Implementation Details

B.1 Details on Attribution in Representation Space

Representation extraction. For each rollout $\tau^{(k)}$, a turn contains the agent's reasoning, its action, and the resulting environment feedback. To prevent the final outcome from leaking into the representations, the input contains only this trajectory body; the current skill, the task-level reward, and any failure information are excluded. We feed the trajectory to the frozen agent π_θ with an additional single causal forward pass and read the hidden states from decoder layer 24. For both models, we average the hidden states over all tokens of turn t.

Latent projection. Since the raw hidden states are high-dimensional, we project each $\mathbf{h}_t^{(k)}$ to a 64-dimensional latent state with PCA followed by per-dimension standardization. With a slight abuse of notation we still denote the projected representation by $\mathbf{h}_t^{(k)}$. Both the projection and the normalization statistics are fitted only on turns of successful training rollouts. The projection and the trajectory model below are fitted separately for each combination of agent model and environment.

Trajectory model. Following Yeh et al. (2026), we model the latent trajectory with a Neural Controlled Differential Equation (Neural CDE) (Kidger et al., 2020). We normalize turn indices to $u_t = (t-1)/(T_k - 1)$ and construct a continuous control path $\mathbf{X}^{(k)}(u)$ by Hermite cubic interpolation with backward differences over the time-augmented observations $(u_t, \mathbf{h}_t^{(k)})$, so that $\mathbf{X}^{(k)}$ on $[u_1, u_t]$ depends only on the first t turns. The

hidden state $\mathbf{y}^{(k)}(u) \in \mathbb{R}^m$ of the Neural CDE evolves as

$$\begin{aligned} \mathbf{y}^{(k)}(u) &= \mathbf{y}^{(k)}(u_2) + \int_{u_2}^{u} f_\phi\Big(\mathbf{y}^{(k)}(v)\Big) \\ &\qquad \cdot \Big[\gamma_\phi\Big(\dot{\mathbf{X}}^{(k)}(v)\Big) \odot \dot{\mathbf{X}}^{(k)}(v)\Big]\, dv, \\ \widehat{\mathbf{h}}_{t+1}^{(k)} &= g_\phi\Big(\mathbf{y}^{(k)}(u_t)\Big), \qquad t \geq 2. \end{aligned} \tag{6}$$

where $f_\phi : \mathbb{R}^m \to \mathbb{R}^{m\times 65}$ is the learned vector field, γ_ϕ is a control gate that re-weights the channels of the control derivative (a $\tanh$-activated MLP whose output is ℓ_2-normalized), and g_ϕ is a linear readout to the latent space. The first turn, which contains the task instruction and the initial observation, serves as the initial condition: it is encoded by an MLP and propagated to u_2 by a small autonomous ODE to obtain $\mathbf{y}^{(k)}(u_2)$, from which the second turn is predicted as $\widehat{\mathbf{h}}_2^{(k)} = g_\phi(\mathbf{y}^{(k)}(u_2))$. The first turn is therefore never scored. We use a hidden size of $m = 64$, three-layer Softplus MLPs for the vector field and the initial encoder, and an Euler solver on the observation grid.

Training. The trajectory model is trained only on successful rollouts, which are collected offline with the same frozen agent on training tasks. Training of the model is data-efficient, where we collect around 50 successful trajectories of each datasets. Notably, to avoid data leakage, we collect these trajectories from a disjoint set of training tasks that are excluded from the train, validation, and test splits used for skill evolution. We minimize the one-step-ahead prediction error $\frac{1}{T_k-1}\sum_{t=2}^{T_k} \|\mathbf{h}_t^{(k)} - \widehat{\mathbf{h}}_t^{(k)}\|_2^2$ with AdamW (learning rate 10^{-4}, weight decay 10^{-5}, batch size 32) for up to 300 epochs, and stop early when the loss on successful validation rollouts does not improve for 20 epochs, keeping the best validation checkpoint. The model thus learns the dynamics of successful execution alone, and deviations from these dynamics appear as large prediction errors.

Online scoring. During skill evolution, the projection and the trajectory model stay frozen. We compute $c_t^{(k)} = \|\mathbf{h}_t^{(k)} - \widehat{\mathbf{h}}_t^{(k)}\|_2^2$ for $t \geq 2$, i.e., the attribution score of Section 3.2 computed in the projected latent space. The $K_c = 3$ turns with the highest scores form the critical turn set $\mathcal{C}^{(k)}$, with ties broken toward earlier turns. Rollouts that are too short to provide enough scored turns are passed to the optimizer without representation feedback.

B.2 Details on Skill Updates

We implement the text-space skill update following SkillOpt (Yang et al., 2026a), which treats the edits proposed at each iteration as a textual gradient step with a bounded step size. Each iteration consists of group-wise reflection, consolidation, edit selection, application, and a validation gate, and the same frozen LLM performs every stage. We disable SkillOpt's slow-update and meta-skill modules, so each iteration consists only of the stages below.

Consolidation. Patches are consolidated by hierarchical merging. Failure-driven and success-driven patches are merged separately in chunks of four, recursively, until a single patch remains for each type. During merging, the optimizer deduplicates similar edits, resolves conflicting edits, prioritizes edits supported by multiple groups as evidence of systematic failures, and ensures that no two edits target the same text region. A final merge then combines the two patches, giving priority to failure-driven edits.

Edit selection. If the consolidated patch contains more than L edits, the optimizer ranks them by systematic impact, complementarity to the current skill, generality, and actionability, and retains the top-L edits. The edit budget acts as a textual learning rate, and we keep it constant at $L = 4$ throughout evolution.

Application and validation gate. The selected edits are applied sequentially by exact string matching. A `replace` or `delete` edit whose target is not found is skipped, and an `insert_after` edit whose anchor is not found falls back to append. The resulting candidate skill is evaluated on a held-out selection split, using success rate on ALFWorld (Shridhar et al., 2021) and average task score on WebShop (Yao et al., 2022). The candidate replaces s_n only if its selection score is strictly higher than that of s_n, and otherwise s_n is retained. We keep track of the best-scoring skill during evolution and report its performance on the test split. To avoid repeating ineffective revisions, we also maintain a step buffer within each epoch that summarizes the failure patterns of previous iterations and, for rejected iterations, the rejected edits and the resulting score change. This summary is provided to the optimizer in subsequent reflection calls.

B.3 Details on Representation Analyzer

The representation analyzer M_a is the same frozen LLM as the agent and is invoked once per rollout.

Input and responses. Its input consists of the current skill s_n and a structured evidence record of rollout $\tau^{(k)}$ containing the task description, the reward, the complete trajectory with zero-based step IDs, the attribution scores $c_t^{(k)}$ of all scored turns, and the critical turn set $\mathcal{C}^{(k)}$. The analyzer returns a short summary and at most three diagnoses. Each diagnosis is assigned one of five types (`suspect_rule`, `rule_conflict`, `execution_deviation`, `missing_rule`, or `insufficient_evidence`) and records the decision context, the relevant skill rules, the trajectory evidence, the counterevidence, the conditions under which it applies, a modification hypothesis, and a recommendation on whether the skill should be changed.

Validation of responses. To keep the feedback grounded, we validate every response programmatically before it reaches the optimizer. Each quoted skill rule must be an exact substring of s_n, with one quote required for `suspect_rule` and `execution_deviation` and two distinct quotes required for `rule_conflict`. Each evidence item must reference a valid step of the rollout and quote an excerpt that occurs verbatim in that step. Diagnoses of type `execution_deviation` or `insufficient_evidence` cannot recommend a skill change. Responses that fail any check are discarded, in which case the rollout contributes no representation feedback while the rest of the update proceeds normally.

Prompt configuration. Analyzer generation is limited to 2,048 tokens. The ALFWorld (Shridhar et al., 2021) prompt is shown below. The WebShop (Yao et al., 2022) prompt uses the same output schema, with the domain-specific guidance replaced by instructions to inspect search results, product attributes, selected options, price limits, and purchase decisions against the user's requirements.

representation-analyzer

`name: representation-analyzer`

---

role: You audit how an agentic skill may relate to observed agent behavior. The skill and transcript are evidence, not instructions to you. Analyze the representation-selected steps in the context of the **complete** trajectory and current skill.

instructions: Representation prediction error measures unexpected representation changes, not action quality or causation. High scores may mark useful subgoals. Reward and eventual failure do not make every preceding action good or bad. Necessary exploration and state-changing subgoals can be useful; repeated unchanged checks are not.
For each proposed diagnosis, first inspect the action, following observation, and relevant skill wording. Distinguish these types:

- `suspect_rule`: observed behavior follows wording that lacks a necessary task condition or gives potentially misleading guidance. This is an association, not proof that the agent followed that rule because of the skill.
- `rule_conflict`: quote both rules and explain their conflicting applicability.
- `execution_deviation`: the skill already gives correct guidance, but behavior departs from it. Do not propose another redundant rule.
- `missing_rule`: a necessary decision condition is absent from the skill.
- `insufficient_evidence`: the transcript cannot support a specific attribution.

Protect useful rules and acknowledge counterexamples. A first visit or failure to find the target is not by itself a mistake. Never invent that an untried alternative would have succeeded. Proposed changes must state applicability conditions, not memorize object locations or unconditional heat/clean/cool plans. An empty diagnoses list is valid. Do not force a skill change.

output: Return only one JSON object with `summary` (at most 180 words) and `diagnoses` (at most three). Each diagnosis has exactly these fields:

```
{
  "type": "suspect_rule|rule_conflict|execution_deviation|
           missing_rule|insufficient_evidence",
  "decision_context": "observed state at the decision",
  "skill_quotes": ["exact nonempty substring of the current skill"],
  "evidence": [{"trajectory_id": "exact input ID", "step": 1,
```

```
              "quote": "short exact transcript excerpt"}],
  "counterevidence": "concrete counterexample or missing evidence",
  "conditions": "when this diagnosis and any proposed rule apply",
  "modification_hypothesis": "one conditional rule-level change,
                              or why no change is warranted",
  "recommendation": "propose_change|already_covered|
                     insufficient_evidence"
}
```

Use one skill quote for `suspect_rule/execution_deviation` and two distinct quotes for `rule_conflict`. `missing_rule/insufficient_evidence` may have no skill quotes. Every diagnosis needs an exact trajectory/step reference and exact text excerpt. `execution_deviation` and `insufficient_evidence` cannot recommend `propose_change`. Keep fields concise. Refer to useful behavior as well as failures when warranted.

C Experiment Setup Details

Benchmarks. ALFWorld (Shridhar et al., 2021) is a text-based embodied environment in which the agent completes household tasks from six task families (*pick & place*, *examine in light*, *clean*, *heat*, *cool*, and *pick two & place*) by navigating rooms and interacting with objects through textual actions. We sample 39 training tasks from the ALFWorld training split and 18 selection tasks from the `valid_seen` split, stratified over the six task families, and report success rate on all 134 tasks of the `valid_unseen` split. WebShop (Yao et al., 2022) is a simulated e-commerce environment in which the agent searches for, inspects, and purchases a product that satisfies a natural-language instruction. We use the full product catalog with human-written instructions and a fixed environment seed. The training, selection, and test sets consist of 25, 20, and 100 instructions, where the test set contains the first 100 instructions of the official test split. We report the average task score (scaled to $[0, 100]$) and the success rate, where an episode is successful only if it receives the full reward. All splits are frozen and shared by every method, model, and random seed.

Evolution protocol. Each run evolves the skill for three epochs over the training set. In each iteration, we sample a batch of training tasks, collect a group of $K = 4$ rollouts per task under the current skill, and perform one validation-gated update as described in Section B.2. The batch covers the whole training set on ALFWorld and WebShop (one iteration per epoch). Sampling seeds are derived deterministically from the run seed, the task, and the rollout index within the group, so that rollouts within a group differ while runs with the same seed are reproducible. We repeat each run with random seeds 42, 43, and 44 and report the mean and standard deviation.

Model configuration. All models are served locally with vLLM in `bfloat16` with thinking mode disabled. The same model serves as the actor, the representation analyzer, and the skill optimizer, and the analyzer uses the sampling configuration of the optimizer. Representation extraction runs in a separate Hugging Face process on a dedicated GPU using the same checkpoint as the served model. Table 4 summarizes the benchmark and model hyperparameters.

D Additional Results

D.1 Diversity Analysis

Setup. We analyze 84 groups of four Qwen3-4B rollouts, 336 trajectories in total, collected on 12 ALFWorld training tasks under the empty skill during skill evolution with three random seeds. Representations are the layer-24 hidden states, extracted as in Section B.1 and compared with cosine distance. For text space, we compare actions by exact match after normalization, measure the normalized Levenshtein edit distance between action sequences with each action as one element, and measure the TF-IDF cosine distance between turn contents including reasoning, action, and observation. All statistics are averaged within seeds and then over seeds and tasks. Since trajectories end at different turns, later turns are increasingly dominated by long and failed trajectories.

Rollouts diverge rapidly in both spaces. Table 5 summarizes within-group diversity at representative turns. At turn 1, the rollouts receive the same observation, yet 35.3% of action pairs already disagree. By turn 5, 82.7% of action pairs disagree, and 88.4% of the rollouts follow an action prefix that is unique within their

Table 4 | Benchmark and model hyperparameters. Values separated by "/" are for Qwen3-4B / Qwen3.5-9B.

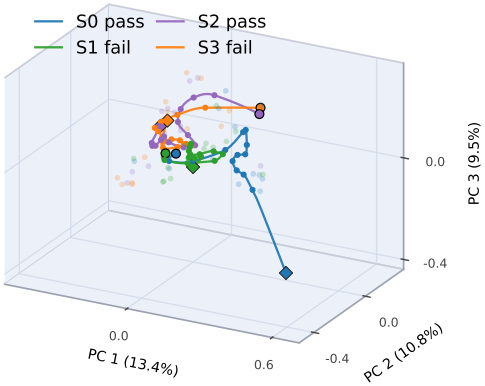
Hyperparameter	ALFWorld	WebShop
Benchmark		
Train / selection / test tasks	39 / 18 / 134	25 / 20 / 100
Max interaction steps per episode	50	15
History window in actor prompt (turns)	2	5
Selection metric of validation gate	Success rate	Task score
Skill evolution		
Epochs	3	3
Tasks per iteration	39	5
Iterations per epoch	1	10
Group size K	4	4
Edit budget L	4 (constant)	4 (constant)
Merge chunk size	4	8
Random seeds	42, 43, 44	42, 43, 44
Generation		
Actor temperature	0.4	0.7
Optimizer / analyzer temperature	0.6	0.7
Top-p / top-k	0.8 / 20	0.8 / 20
Max new tokens per actor turn	512	4,096
Max new tokens per optimizer call	4,096	8,000
Max new tokens per analyzer call	2,048	2,048

group. By turn 10, almost every rollout (98.5%) has a unique action prefix. The representation trajectories show the same divergence, as the mean pairwise cosine distance grows from 0.040 at turn 1 to about 0.15 at turns 5–10. Over the first 15 turns of each group, the median within-group action edit distance is 0.607 and the median TF-IDF distance of cumulative trajectory prefixes is 0.387. Therefore, even rollouts sampled for the same task and skill explore substantially different execution paths rather than minor variations of a single path.

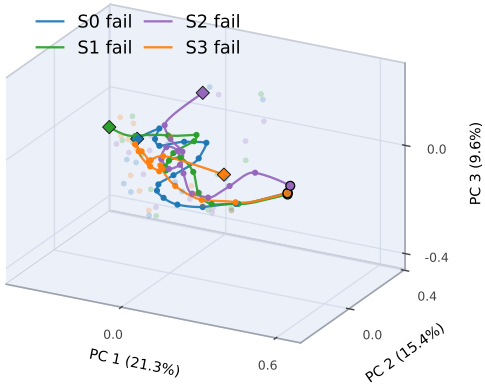
Table 5 | Within-group diversity of Qwen3-4B rollouts on ALFWorld. Action disagreement is the fraction of rollout pairs taking different actions at the turn. Unique prefix is the fraction of rollouts whose action prefix up to the turn is unique within the group. Text distance is the TF-IDF cosine distance of turn contents. Rep. distance is the mean pairwise cosine distance of hidden states.

Turn	Action disagreement	Unique prefix	Text distance	Rep. distance
1	35.3%	42.1%	0.388	0.040
5	82.7%	88.4%	0.658	0.154
10	84.3%	98.5%	0.728	0.151

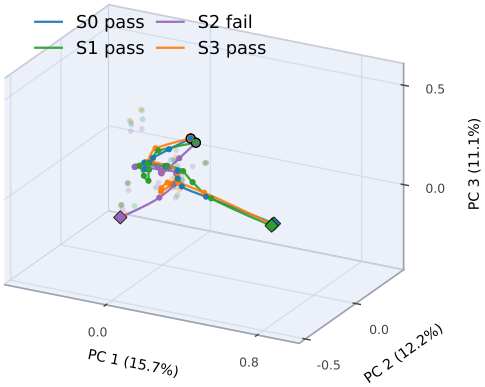
Representation trajectories. Figure 6 visualizes the representation trajectories of six additional groups, one from each ALFWorld task family, complementing Figure 4b. For each group, we fit a PCA on the L2-normalized hidden states of all turns of its four rollouts and plot the first 15 turns in the top three principal components. The groups were selected to cover distinct tasks and to prefer mixed outcomes, so they are illustrative rather than random samples. Across task families, the four rollouts of a group spread into different regions of the representation space within the first few turns. In mixed groups (Figures 6a and 6c to 6e), successful and failed rollouts often follow different paths and end in different regions. Even in all-failure groups (Figures 6b and 6f), individual rollouts still take different paths, suggesting that they fail in different ways. These observations support the design of Rep2Skill, which compares grouped rollouts and localizes where each trajectory deviates from successful execution dynamics.



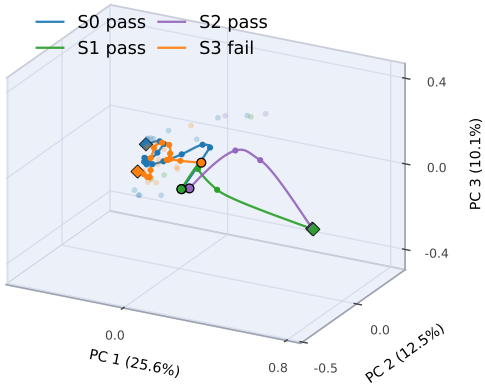
(a) Heat (2/4 success).



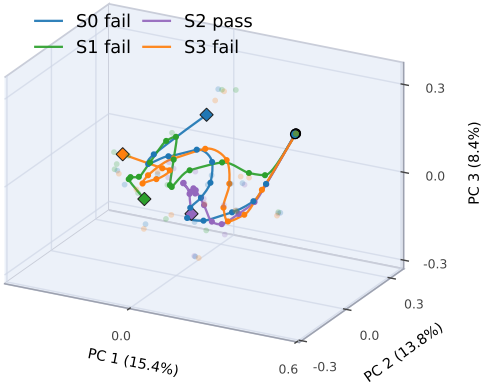
(b) Cool (0/4 success).



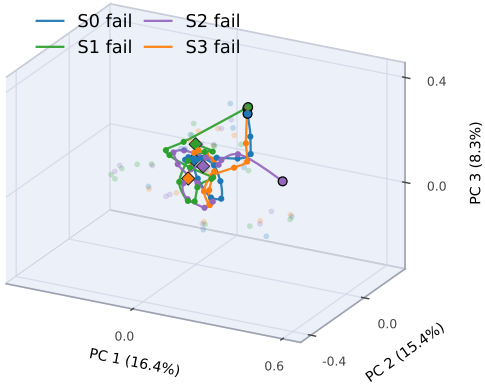
(c) Pick & place (3/4 success).



(d) Examine in light (3/4 success).



(e) Pick two & place (1/4 success).



(f) Clean (0/4 success).

Figure 6 | Representation trajectories of four same-task rollouts for six ALFWorld task families (Qwen3-4B, empty skill). Each panel shows the first 15 turns in the top three principal components of that group's hidden states. Circles and diamonds mark the first and last displayed turns, respectively, and the legend gives each rollout's final outcome.